

# Techniques de compilation

Auditoire : CPI-2

Année Universitaire : 2025-2026

Code Classroom: **6xqrgwgl**

**Dr.-Ing. Rakia Saidi**

Maitre assistante en informatique

[rakia.saidi@isimg.tn](mailto:rakia.saidi@isimg.tn)

# Plan de ce cours

---

- **Chapitre 1**: Introduction à la compilation
- **Chapitre 2**: Rappels à la théorie des langages
- **Chapitre 3**: Analyse lexicale
- **Chapitre 4**: Analyse syntaxique
- **Chapitre 5**: Analyse sémantique
- **Chapitre 6**: Génération de code

## Chapitre 5 :Analyse sémantique

- ▶ En compilation, l'*analyse sémantique* est l'étape qui suit l'*analyse syntaxique*.
- ▶ On peut la considérer comme la dernière partie du “front end” du compilateur, puisque c'est potentiellement la dernière qui est spécifique au langage source.
- ▶ Le rôle est de comprendre et vérifier le *sens* du code source, qui est défini par la sémantique du langage.

- Cette phase répond à la question suivante:

**Est-ce que l'assemblage des constituants  
du programme a un sens?**

**Exemple 1.5:** on vérifie s'il n'y a pas d'erreur de typage.

```
....  
int x ;  
char c;  
x=2 ;  
X=X + c;  
....
```

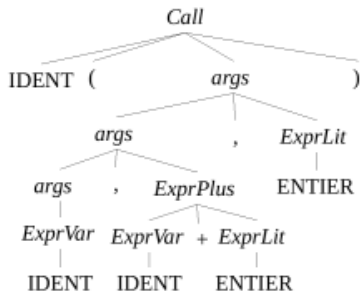
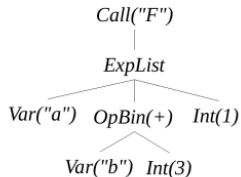
Instruction sémantiquement  
correcte

Correct pour les 2 premières phases  
et sémantiquement faux (typage).

- ▶ Le but de l'analyse sémantique est de produire un *arbre de syntaxe abstraite* (AST) sémantiquement valide.
- ▶ Cela signifie qu'on doit pouvoir interpréter ou compiler le code représenté par cet AST sans se soucier de ce qui a déjà été vérifié.
- ▶ Selon les langages, ces vérifications sont plus ou moins importantes.

Arbre de syntaxe abstrait (AST)

- ▶ Forme porteuse de la sémantique
- ▶ Intègre les valeurs sémantiques des tokens
- ▶ Supprime les tokens inutiles : « sucre syntaxique » (séparateurs, parenthèses...)
- ▶ Se libère du « tyran algébrique » : « OPBIN » vs. « {PLUS,MOINS,...} »
- Exemple : « F(a, b + 3, 1) »

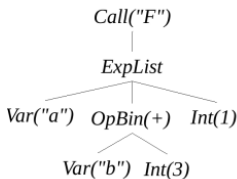
**AST**

## Arbre sémantique

Un arbre sémantique est un AST (arbre de syntaxe abstraite) enrichi avec des informations de sens, principalement :

- ▶ les types (int, float, bool, ...)
- ▶ les liaisons (variables → déclarations)
- ▶ les vérifications (compatibilité des opérations)

### AST



### Hypothèses sémantiques (nécessaires)

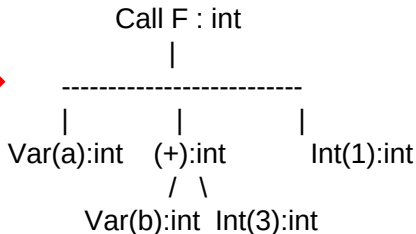
Pour construire l'arbre sémantique, on suppose :

a:int

b:int

F:(int,int)→int

### Arbre sémantique



### Ce que montre cet arbre sémantique

- ✓ propagation des types
- ✓ vérification des opérations
- ✓ validation de l'appel de fonction
- ✓ base pour génération de code



## Portée( scope)

- ▶ La portée d'une variable est la zone du programme où cette variable est visible et utilisable.
- ▶ la portée (scope) est un concept fondamental en sémantique.

### ▶ Types de portée

#### 1. Portée globale

définie hors fonctions  
accessible partout

Python:

```
x = 5
```

```
def f():
```

```
    print(x)
```

x est visible dans toute la fonction, elle est globale

#### 2. Portée locale

définie dans une fonction/bloc  
accessible uniquement dedans

Python:

```
def f():
```

```
    x = 10
```

```
    print(x)
```

```
print(x) # ✗ erreur
```

x dans f est locale, elle n'existe pas en dehors

## Vocabulaire : “statique” vs “dynamique”

- ▶ On dit que quelque chose est *statique* quand on peut le déterminer juste à partir du code source, sans avoir besoin d'exécution.
- ▶ On dit que quelque chose est *dynamique* si il ne peut être déterminé qu'à l'exécution.
- ▶ Au moment de la compilation, on est donc sûr de l'*analyse statique*.

## Types de vérifications

- \* Quels types de vérifications peuvent être réalisés lors de l'analyse sémantique ?

## Types de vérifications

- \* Quels types de vérifications peuvent être réalisés lors de l'analyse sémantique ?
  - ▶ Le minimum est de vérifier l'utilisation des noms.
    - La façon correcte de le faire dépend du langage.

## Types de vérifications

- \* Quels types de vérifications peuvent être réalisés lors de l'analyse sémantique ?
  - ▶ Le minimum est de vérifier l'utilisation des noms.
    - La façon correcte de le faire dépend du langage.
  - ▶ Le reste de l'analyse sémantique consiste essentiellement au *typage*.

## Environnements

- ▶ Pour faire cette analyse, il est nécessaire de maintenir un *environnement*.
- ▶ Il peut simplement s'agir d'une liste des variables accessibles au point du programme en train d'être analysé\*.
- ▶ Cependant attention aux spécificités du langage :
  - il peut être nécessaire de distinguer l'environnement local de l'environnement global.

## Environnements

- ▶ Pour faire cette analyse, il est nécessaire de maintenir un *environnement*.
- ▶ Il peut simplement s'agir d'une liste des variables accessibles au point du programme en train d'être analysé\*.
- ▶ Cependant attention aux spécificités du langage :
  - il peut être nécessaire de distinguer l'environnement local de l'environnement global.

- ▶ Le *typage* est l'activité principale de l'analyse sémantique.
- ▶ Il s'agit de vérifier que le programme est correctement typé, par exemple :
  - que les fonctions reçoivent le bon nombre d'arguments,
  - que les types des arguments sont bien ceux attendus par la fonction,
  - que les types des valeurs affectées correspondent à ceux des variables,
  - ...
- ▶ Encore une fois, les propriétés du langage vont fortement influencer ce qu'on va devoir/pouvoir faire.



- ▶ Les langages *dynamiquement typés* attribuent des types aux *valeurs* mais pas aux *variables*.
- ▶ Le type des variables ne peut donc pas toujours être statiquement connu.

- ▶ Les langages *statiquement typés* attribuent des types aux valeurs et aux *variables*.
- ▶ Cela permet plus de vérifications statiques.

- Les langages *faiblement typés* autorisent les valeurs à changer de type automatiquement quand c'est nécessaire et possible.

- Les langages *fortement typés* n'autorisent pas les valeurs à changer de types.

## Typage dynamique faible

- ▶ Commençons par la combinaison qui permet le moins de vérification...
- ▶ Celle qui ne devrait donc surtout pas être utilisée dans les environnements critiques...
- ▶ Comme par exemple là où tout se fait aujourd'hui... le navigateur.
- ▶ Hé oui... **JavaScript !** -\_-'
  - ```
foo = "coucou";  
foo = 42;  
console.log("foo vaut " + foo);
```

## Typage dynamique fort

### ► Python.

- ```
foo = "coucou"
foo = 42
print("foo vaut " + foo) # erreur à l'exécution
print("foo vaut " + str(foo)) # on doit explicitement demander la conversion
```

Typage statique faible

C.

- ```
int foo = 42;  
foo = "coucou"; // erreur à la compilation  
double x = foo; // conversion int vers double automatique
```

- ▶ Cette fois, ils associent à chaque variables des informations sur son type permettant de vérifier non seulement que la variable existe dans le contexte mais aussi qu'elle est utilisée correctement :
  - variables : type ;
  - fonctions : arité, types attendus en argument, types de retour.



- ▶ L'idée générale est de vérifier que les informations de types qu'on possède sont compatibles avec celles qu'on doit deviner et avec le système de types du langage.
- ▶ On va faire cela en construisant un *arbre d'inférence*.
- ▶ Cette arbre correspond en fait à des étiquettes d'information sur les nœuds de notre AST.
- ▶ Il va servir à :
  - deviner le type de certaines expressions faisant appel à des fonctions polymorphes (ajout de contraintes, inférences),
  - produire des erreurs plus ou moins précises,
  - rajouter du code pour la conversion de type dans les langages faiblement typés.
  - montre étape par étape comment on déduit le type d'une expression à l'aide de règles de typage.

$F(a, b+3, 1)$ 

## Règles utilisées:

- Constante :

$$\frac{}{n : int}$$

- Variable :

$$\frac{}{x : T}$$

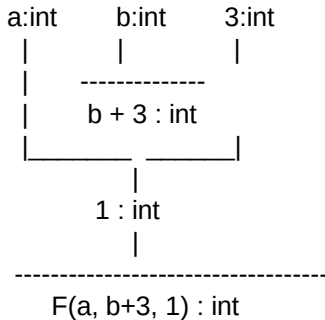
- Addition :

$$\frac{e_1 : int \quad e_2 : int}{e_1 + e_2 : int}$$

- Appel de fonction :

$$\frac{e_1 : T_1 \quad e_2 : T_2 \quad e_3 : T_3}{F(e_1, e_2, e_3) : int}$$

## Arbre d'inférences



- $b$  et  $3$  sont  $int$
- donc  $b + 3 : int$
- tous les arguments sont  $int$
- donc :

 $F(a, b + 3, 1) : int$

Toujours :

- Vérifier si le mot est **accepté**
- Ensuite construire :
  1. AST : structure du programme
  2. arbre sémantique : sens + vérifications
  3. arbre d'inférence: preuve formelle du typage

# Techniques de compilation

Auditoire : CPI-2

Année Universitaire : 2025-2026

Code Classroom: **6xqrgwgl**

**Dr.-Ing. Rakia Saidi**

Maitre assistante en informatique

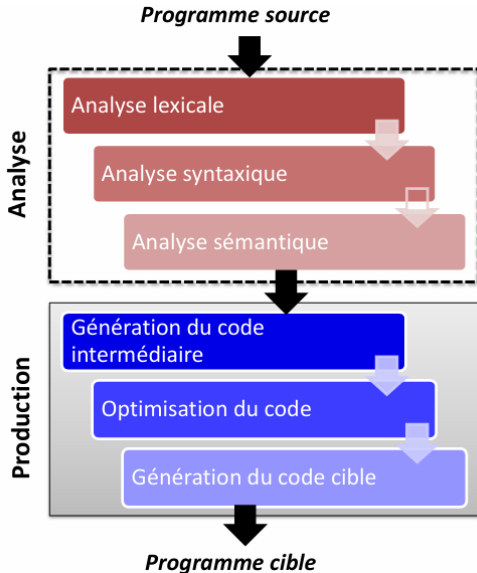
[rakia.saidi@isimg.tn](mailto:rakia.saidi@isimg.tn)

## Chapitre 6 :Génération de code

# Plan de ce cours

---

- **Chapitre 1**: Introduction à la compilation
- **Chapitre 2**: Rappels à la théorie des langages
- **Chapitre 3**: Analyse lexicale
- **Chapitre 4**: Analyse syntaxique
- **Chapitre 5**: Analyse sémantique
- **Chapitre 6**: Génération de code



La **phase de production** est la deuxième grande phase de la compilation.

### Rôle

Transformer la représentation intermédiaire en programme cible.

### Elle comprend :

1. Génération du code intermédiaire
2. Optimisation du code
3. Génération du code cible

Représentation intermédiaire



Optimisation



Code cible (assembleur /  
machine)



Le **code intermédiaire** est une représentation du programme source indépendante de l'architecture machine.

### Objectifs:

- Faciliter l'optimisation du code
- Assurer la portabilité du compilateur
- Simplifier la génération du code cible

### Caractéristiques

- Indépendant du processeur
- Proche du langage machine mais abstrait
- Facile à manipuler et à transformer

Prenons l'exemple:

```
a = b + c;
```

### Objectif

Représenter le programme sous une forme **abstraite indépendante de la machine**

### Code à trois adresses

```
t1 := b + c
```

```
a := t1
```

- t1 : variable temporaire
- Une opération par instruction
- Structure simple pour faciliter la suite (optimisation, traduction)

Objectif: améliorer performance et réduire coût (Réduire le nombre d'instructions)

### Après optimisation

$a := b + c$

### Explication

- Suppression de la variable temporaire t1
- Code plus direct et efficace

### Allocation des registres



#### Objectif

Associer les variables à des registres

$b \rightarrow R1$

$c \rightarrow R2$

résultat  $\rightarrow R1$

La **génération du code cible** consiste à traduire le code intermédiaire en un code exécutable par la machine (code machine ou assembleur).

Le **langage assembleur** est un langage de bas niveau :

- proche du langage machine
- dépendant du processeur
- composé d'instructions symboliques (MOV, ADD, MUL...)

MOV b, R1 ; charger b dans R1

ADD c, R1 ;  $R1 = b + c$

MOV R1, a ; stocker le résultat dans a

➔ Il est ensuite traduit en **code machine binaire**.